

EXPERIMENT 6

IMPLEMENT ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM

To implement error detection and correction at the Data Link Layer using the Hamming Code method by converting data into binary, adding parity bits, identifying and correcting single-bit errors, and retrieving the original data.

SENDER ALGORITHM

1. Accept the original input data.
2. Convert the given data into binary form.
3. Determine the required number of parity bits using:

$$2^r \geq m + r + 1$$

4. Create the required bit positions from 1 to n.
5. Insert parity bits at positions:

$$1, 2, 4, 8, \dots$$

6. Place the actual data bits in the remaining positions.
7. Calculate the parity bits using the **even parity** method.
8. Generate the final Hamming code.
9. Transmit or store the encoded data through the communication channel.

RECEIVER ALGORITHM

1. Receive the Hamming code from the communication channel.
2. Verify each parity group.
3. Calculate the parity-check values **P1, P2, P4, P8, ...**
4. Combine the obtained parity-check values.
5. Calculate the **syndrome**.
6. If the syndrome is **0**, there is no error.
7. If the syndrome is **not 0**, it indicates the position of the erroneous bit.
8. Change the value of the bit at the detected position.
9. Remove the parity bits from the corrected code.
10. Retrieve the original data.

PROGRAM

```
#include <stdio.h>

void encode(int data[4], int code[7])
{
    int p1, p2, p4;

    code[2] = data[0];
    code[4] = data[1];
    code[5] = data[2];
    code[6] = data[3];

    p1 = code[2] ^ code[4] ^ code[6];
    p2 = code[2] ^ code[5] ^ code[6];
    p4 = code[4] ^ code[5] ^ code[6];

    code[0] = p1;
    code[1] = p2;
    code[3] = p4;
}

void decode(int code[7])
{
    int p1, p2, p4;
    int errorPosition;

    p1 = code[0] ^ code[2] ^ code[4] ^ code[6];
    p2 = code[1] ^ code[2] ^ code[5] ^ code[6];
    p4 = code[3] ^ code[4] ^ code[5] ^ code[6];

    errorPosition = p4 * 4 + p2 * 2 + p1;

    if (errorPosition == 0)
    {
        printf("No error detected.\n");
    }
    else
    {
        printf("Error detected at position: %d\n", errorPosition);
        code[errorPosition - 1] ^= 1;
        printf("Error corrected successfully.\n");
    }

    printf("Corrected Hamming Code: ");

    for (int i = 0; i < 7; i++)
        printf("%d", code[i]);

    printf("\n");

    printf("Original Data: %d%d%d%d\n",
           code[2], code[4], code[5], code[6]);
}

int main()
{
    int data[4];
    int code[7];

    printf("Enter 4-bit data: ");
```

```

    for (int i = 0; i < 4; i++)
        scanf("%d", &data[i]);

    encode(data, code);

    printf("\nEncoded Hamming Code: ");

    for (int i = 0; i < 7; i++)
        printf("%d", code[i]);

    printf("\n");

    code[5] ^= 1;

    printf("Received Code with Error: ");

    for (int i = 0; i < 7; i++)
        printf("%d", code[i]);

    printf("\n");

    decode(code);

    return 0;
}

```

The program structure and logic are based on the uploaded experiment, including the 4-bit data, 7-bit Hamming code, parity calculation, intentional single-bit error, and correction process.

RESULT

Thus, the Hamming Code technique was successfully implemented to detect and correct a single-bit error at the Data Link Layer.

